

UNIT 3

Project Monitoring

Project: AI-Based Simulation System for Reconstructing Ancient Indian Bead Numeration Algorithms

3.1 Experiment 1: Project Performance Analysis

3.1.1 Cost, Time, Scope and Performance Monitoring

Project performance monitoring is the continuous process of measuring actual progress against established baselines for cost, schedule, scope, and quality, and then taking corrective action when deviations are identified. For this project, Earned Value Management (EVM), sprint reviews, milestone tracking, and test monitoring were used to monitor performance.

3.1.2 Key Performance Indicators (KPIs)

KPI	Target	Measurement Method
Schedule Variance (SV)	> -0.5 Lakhs	EVM — EV minus PV
Cost Variance (CV)	> -0.5 Lakhs	EVM — EV minus AC
Schedule Performance Index (SPI)	≥ 0.90	EV / PV
Cost Performance Index (CPI)	≥ 0.90	EV / AC
Defect Density	< 3 per module	Code review and unit testing
Requirement Stability Index (RSI)	$\geq 90\%$	(Initial requirements – Changes) / Initial
AI Module Response Accuracy	$\geq 85\%$	Gemini output validation tests
Module Completion Rate	100% by Day 85	Sprint velocity tracking

3.1.3 Earned Value Analysis (EVA) — Month 2 Status

Parameter	Symbol	Value (INR Lakhs)
Budget at Completion	BAC	10.00
Planned Value (cumulative)	PV	4.20
Earned Value (cumulative)	EV	3.60
Actual Cost (cumulative)	AC	4.50

3.1.4 Variance Analysis

Cost Variance (CV) = $EV - AC = 3.60 - 4.50 = -0.90$ Lakhs, which shows that the project was over budget at the Month 2 checkpoint. Schedule Variance (SV) = $EV - PV = 3.60 - 4.20 = -0.60$ Lakhs, which indicates that the project was behind schedule at the same checkpoint.

3.1.5 Cost and Schedule Performance Indices

Cost Performance Index (CPI) = $EV / AC = 3.60 / 4.50 = 0.80$. This means that for every INR 1.00 spent, only INR 0.80 worth of work was completed. Schedule Performance Index (SPI) = $EV / PV = 3.60 / 4.20 = 0.857$, meaning the project was progressing at 85.7% of the planned rate.

3.1.6 Forecasting

Estimate at Completion (EAC) = $BAC / CPI = 10.00 / 0.80 = 12.50$ Lakhs. Estimate to Complete (ETC) = $EAC - AC = 8.00$ Lakhs. To-Complete Performance Index (TCPI) = $(BAC - EV) / (BAC - AC) = 1.164$, which indicates that the remaining work must be completed more efficiently than the work completed so far in order to remain within the original budget.

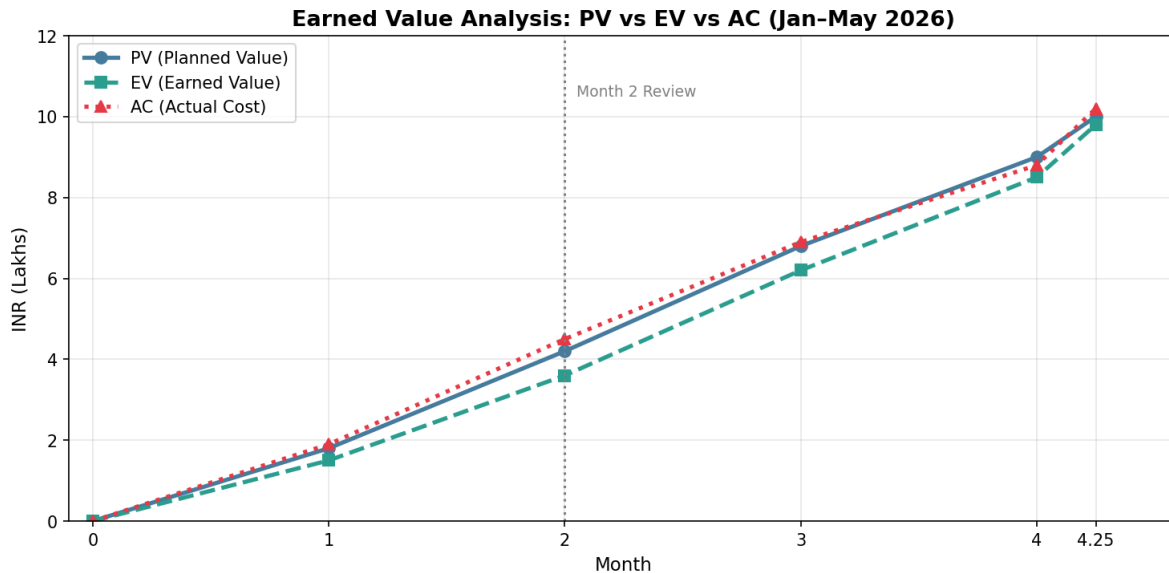


Figure 3.1: Earned Value Analysis — PV vs EV vs AC

3.1.7 Performance Improvement Plan

Action	Expected Impact	Cost
Switch Gemini API to batch request mode	Reduce API costs by ~30%; improve CPI	INR 0
Parallelize Activities E and F	Recover 3–4 days of schedule slip	INR 0
Use INR 1 Lakh contingency reserve	Buffer against EAC increase	INR 1 Lakh
Introduce weekly EVM check-ins	Earlier detection of variances	INR 0

3.1.8 Project Controlling Techniques

Cost control is managed through EVM metrics, API usage budget alerts, and contingency reserve management. Schedule control is maintained through critical path monitoring, sprint velocity analysis, milestone dashboards, and float monitoring. Scope control is managed via the Requirements Traceability Matrix and formal change control review.

3.1.9 Project Change Control

Step	Description
1. Change Request	Formal change request submitted via GitHub Issues
2. Impact Analysis	PM assesses impact on scope, time, cost, and quality
3. CCB Review	Team reviews the request in a scheduled meeting
4. Decision	Approve, reject, or defer with justification
5. Implementation	Approved change is developed in a new feature branch
6. Verification	QA confirms correct implementation without regression

3.1.10 Project Quality Controlling Techniques

Technique	Application	Frequency
Code Reviews	Peer review of Next.js and Firebase code via PRs	Per commit
Unit Testing	Automated Jest test suite for components	Per sprint
Integration Testing	API endpoint and module interaction testing	Per sprint
AI Output Validation	Review of Gemini-generated narrations	Weekly
Performance Testing	Lighthouse score monitoring	Per release
Cross-Browser Testing	Chrome, Firefox, Edge	Per release

3.1.11 Revised Forecast After Corrective Actions

Parameter	Before Correction	After Correction
EAC (INR Lakhs)	12.50	11.20
Projected Completion Delay	~5 days	~2 days
CPI (forward-looking)	0.80	0.91
SPI (forward-looking)	0.857	0.94

3.2 Experiment 2: Risk Management and Mitigation

3.2.1 Project Risk Analysis

Risk management is the systematic process of identifying, analyzing, prioritizing, and responding to uncertainties that may impact the project. Six major risks were identified for this project based on technical, financial, and team-related uncertainties.

Risk ID	Risk Description	Probability (P)	Impact (I)	Risk Score (P×I)
R01	Gemini API rate limits causing response failures under load	0.60	0.80	0.48
R02	AI/Gemini module takes longer than estimated	0.50	0.80	0.40
R03	Firebase Blaze plan costs exceed budget	0.50	0.50	0.25
R04	Complex UI delays abacus simulator development	0.40	0.50	0.20
R05	Low student engagement due to presentation issues	0.30	0.60	0.18
R06	Team availability disruption due to academics	0.30	0.70	0.21

3.2.2 Risk Matrix

Probability / Impact	Low Impact (<0.4)	Medium Impact (0.4–0.6)	High Impact (>0.6)
High Probability (>0.5)	—	R03	R01, R02
Medium Probability (0.3–0.5)	—	R04	—
Low Probability (<0.3)	—	R05	R06

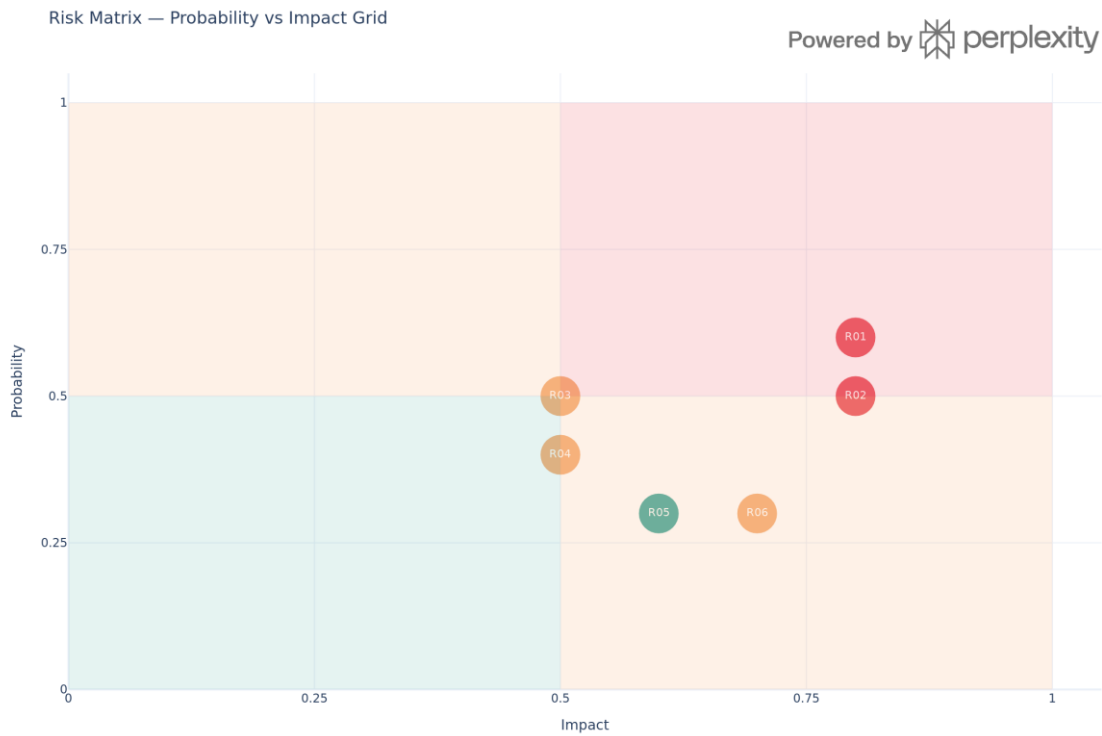


Figure 3.2: Risk Probability-Impact Matrix

3.2.3 Risk Response Strategies

Risk ID	Strategy	Specific Actions	Owner
R01	Mitigate	Caching, batch processing, API quota alerts	Rudransh Yadav
R02	Mitigate	Break AI module into sub-tasks; fallback static content	Rudransh Yadav
R03	Mitigate	Use Firebase Spark for dev; budget alerts	Madhav Nehra
R04	Mitigate	Use a React abacus library base; limit custom animation	Madhav Nehra
R05	Accept + Contingency	User testing and feedback-driven improvements	Rudransh Yadav
R06	Mitigate	Task buffers and cross-training	Ashray Singh

3.2.4 Risk Contingency Reserve

Risk Category	Contingency Allocated (INR Lakhs)	Notes
Gemini API overages (R01, R02)	0.50	Reserved for API billing overruns
Firebase cost overrun (R03)	0.25	Buffer for Blaze plan upgrades
Schedule recovery (R02, R04)	0.50	Overtime/freelancer support if critical
User adoption (R05)	0.25	Content redesign and UX improvements
Unforeseen risks	0.50	General contingency buffer
Total	2.00	~20% of baseline budget

3.3 Experiment 3: RMMM Plan, Configuration Management & GitHub

3.3.1 RMMM Plan

Activity	Frequency	Responsible
Risk review meeting	Weekly	Ashray Singh
Risk register update	Bi-weekly	Risk Owner
Mitigation tracking	Weekly	Ashray Singh
Gemini API quota monitoring	Daily	Rudransh Yadav
Early warning review	Monthly	All Team Members

3.3.2 Risk Triggers and Thresholds

Risk ID	Trigger	Response Activation Threshold
R01	Gemini quota reaches 80% within a sprint	Activate caching immediately
R02	AI module slips by > 3 days	Crash activity D
R03	Firebase bill exceeds INR 3,000	Switch non-critical reads to cache
R04	Abacus UI slips by > 2 days	Use simpler library component
R05	Beta user satisfaction < 60%	Redesign content flow
R06	Any team member unavailable > 3 days	Redistribute tasks

3.3.3 Configuration Items under Control

The configuration management plan covers the Next.js source code, Firebase configuration files, Gemini API integration module, environment variables, educational content files, test suite, project documentation, and Docker configuration. All these artifacts are versioned, controlled, and tracked through GitHub.

3.3.4 GitHub Repository Structure

/bead-numeration-simulator/ contains .github/workflows for CI/CD, src/components and pages for frontend modules, src/lib for gemini.js and firebase.js, public/content for educational JSON, tests for Jest suites, docs for project reports, and Docker-related deployment files.

3.3.5 Git Flow Branching Strategy

Branch	Purpose	Protection Rules
main	Production-ready code deployed to Vercel	PR required; CI pass; 2 approvals
develop	Integration branch for completed features	PR required; CI pass; 1 approval
feature/*	Individual feature development branches	No protection; deleted after merge
release/*	Release preparation and bug fixes	PR to both main and develop

Branch	Purpose	Protection Rules
hotfix/*	Urgent production bug fixes	PR to main; 1 approval

3.3.6 CI/CD Pipeline Stages

Stage	Tool	Trigger
Lint	ESLint + Prettier	On push to any branch
Unit Test	Jest (min. 80% coverage)	On push to feature/develop
Build	Next.js build	On PR to develop
Deploy (Preview)	Vercel Preview	On PR to develop
Deploy (Production)	Vercel Production	On merge to main

3.3.7 Versioning Convention

Semantic Versioning (SemVer) was used: Major.Minor.Patch. Examples include v0.1.0 for the early alpha, v0.3.0 for the beta with AI and abacus integration, and v1.0.0 for the final production release.

3.4 Experiment 4: Unit Testing with Test Cases

3.4.1 Testing Strategy Overview

Test Level	Framework	Target Coverage
Unit Testing	Jest + React Testing Library	80% minimum
Integration Testing	Jest + Firebase Emulator	90% of API endpoints
Regression Testing	Jest + GitHub Actions CI	100% of prior passing tests
AI Output Validation	Custom script + manual review	85% accuracy
Performance Testing	Lighthouse CI	Page load ≤ 3 sec; Accessibility ≥ 90

3.4.2 Sample Test Case 1: Abacus Addition Operation

Field	Details
Test Case ID	TC-ABACUS-001
Test Description	Validate abacus addition: 3 + 4 = 7
Input	Bead position: Rod 1 = 3 beads up; Rod 2 = 4 beads up
Expected Output	Result display: 7; correct bead positions
Actual Output	Result display: 7; beads correctly positioned
Status	PASS
Remarks	Core arithmetic function verified

3.4.3 Sample Test Case 2: Number System Conversion

Field	Details
Test Case ID	TC-NUMCONV-001
Test Description	Validate decimal to binary conversion for input 10
Input	Decimal: 10
Expected Output	Binary: 1010; Hexadecimal: A; Octal: 12
Actual Output	Binary: 1010; Hexadecimal: A; Octal: 12
Status	PASS
Remarks	All conversion outputs verified

3.4.4 Sample Test Case 3: AI Narration Module — Fail Case

Field	Details
Test Case ID	TC-AI-003
Test Description	Validate Gemini narration for “History of Zero in India”
Input	Prompt requesting age-appropriate historical story
Expected Output	Coherent, accurate narration in < 5 seconds
Actual Output	Narration generated but response time was 8.3 sec
Status	FAIL
Root Cause	Gemini API cold start latency
Fix	API warm-up ping every 4 minutes

3.4.5 Test Execution Summary

Module	Test Cases	Passed	Failed	Pass Rate
Abacus Simulator	20	18	2	90.0%
Number System Explorer	15	14	1	93.3%
AI Story Module	14	10	4	71.4%
Quiz Module	14	12	2	85.7%
Auth & Access	9	8	1	88.9%
Integration	7	6	1	85.7%
Total	79	68	11	86.1%

3.4.6 Defect Analysis

Severity	Description	Count	Action
Critical	AI module response time SLA violation	2	Fix immediately
Major	Abacus carry-over logic error	3	Fix within current sprint
Minor	UI alignment issues on mobile	4	Fix in next sprint
Trivial	Tooltip spelling errors	2	Fix during documentation sprint

3.4.7 Corrective Actions

Critical defects were addressed by implementing a warm-up endpoint and caching common Gemini prompts. Major abacus carry-over issues were resolved by rewriting the bead state logic using React useReducer. Full regression testing was planned after the fixes.

3.4.8 Quality Metrics Dashboard

Metric	Target	Current	Status
Overall Test Pass Rate	≥ 95%	86.1%	Below Target
Code Coverage	≥ 80%	74%	Below Target
AI Narration Response Time	< 5 seconds	8.3 sec cold / 3.1 sec warm	Partial
Page Load Time	< 3 seconds	2.4 seconds	On Track
Critical Defects Open	0	2	Immediate Fix Needed
Mean Time to Fix (Critical)	< 24 hours	18 hours	On Track

3.4.9 Conclusion for Unit 3

Unit 3 focused on project monitoring through EVM, risk management, RMMM planning, configuration management, and unit testing. The Month 2 checkpoint showed CPI = 0.80 and SPI = 0.857, confirming cost and schedule challenges mainly caused by Gemini integration complexity. Risk analysis identified six major risks, all with documented responses. Configuration management through GitHub, Git Flow, and CI/CD ensured control of project artifacts. Unit testing revealed 79 test cases with an 86.1% pass rate at the checkpoint, providing a strong basis for corrective action and quality improvement toward final release.

Unit 3 Figures

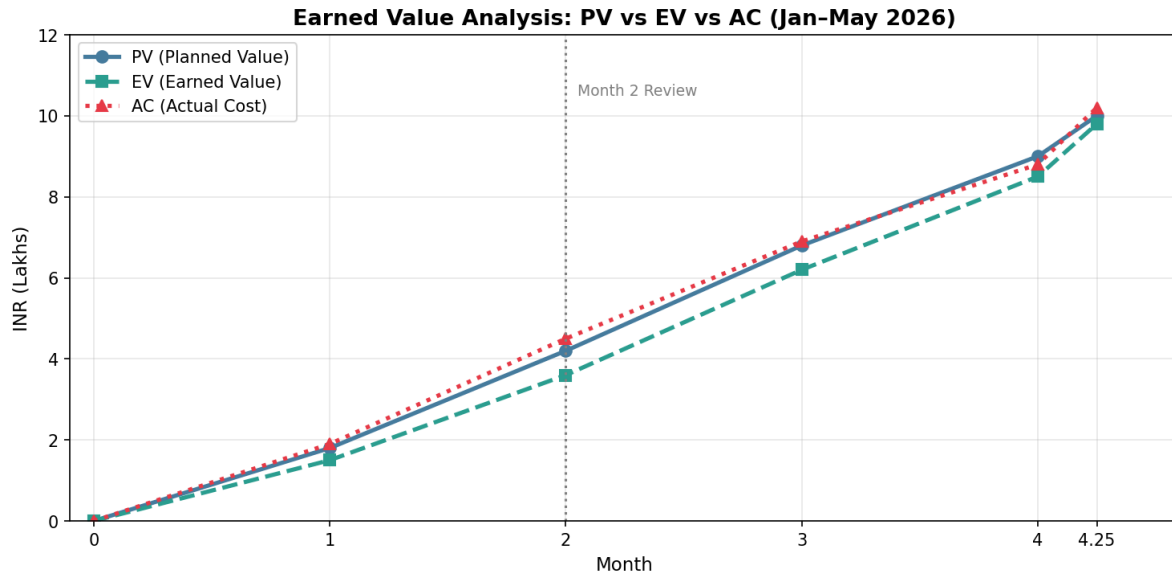


Figure 3.1: Earned Value Analysis — PV vs EV vs AC

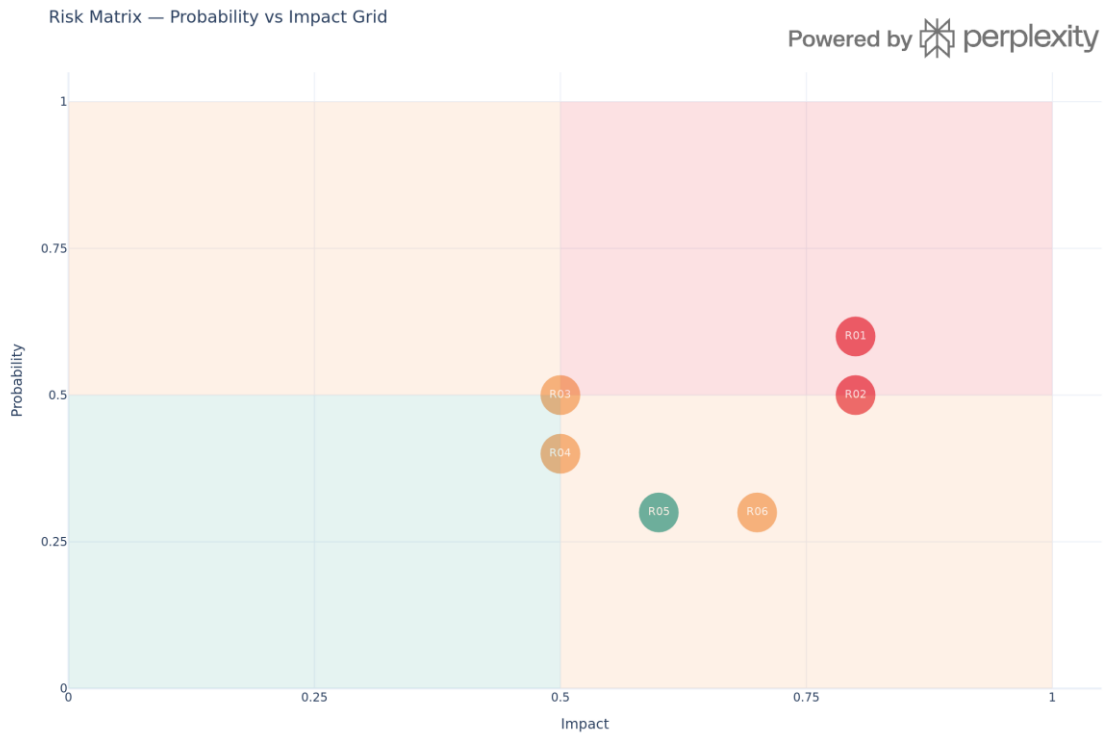


Figure 3.2: Risk Probability-Impact Matrix

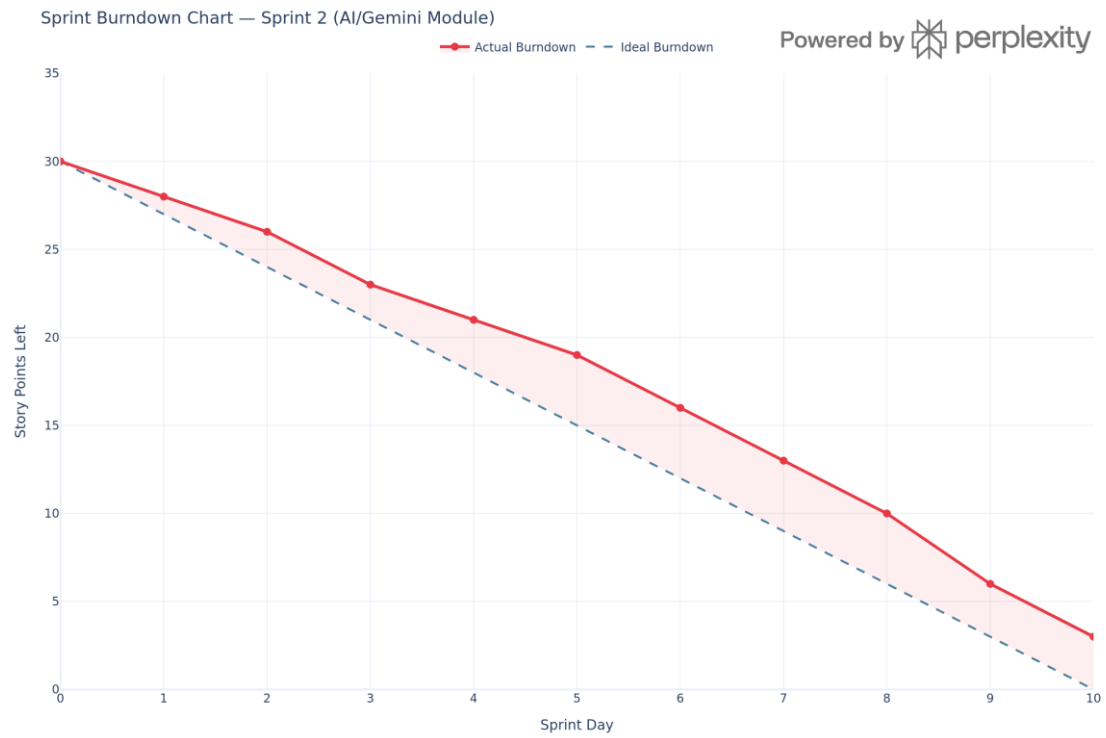


Figure 3.3: Sprint Burndown Chart